

Curriculum for Reinforcement Learning

This PDF conversion (v1) was generated on October 10, 2025*

Online version:

<https://lilianweng.github.io/posts/2020-01-29-curriculum-rl/>

Date: **Estimated Reading Time:** min **Author:** Lilian Weng

⁰This file was prepared by Sakura (bili_sakura@zju.edu.cn). Please contact for any issues or copyright concerns. The original source of this file is <https://lilianweng.github.io/posts/2020-01-29-curriculum-rl/>.

Contents

1 Curriculum for Reinforcement Learning	3
2 Task-Specific Curriculum#	4
3 Teacher-Guided Curriculum#	6
4 Curriculum through Self-Play#	7
5 Automatic Goal Generation#	9
6 Skill-Based Curriculum#	11
7 Curriculum through Distillation#	13
8 Citation#	15
9 References#	15
10 Citation	16
11 References	17

[Lil'Log](#)

- |
- [Posts](#)
- [Archive](#)
- [Search](#)
- [Tags](#)
- [FAQ](#)

1 Curriculum for Reinforcement Learning

Date: January 29, 2020 | Estimated Reading Time: 24 min | Author: Lilian Weng
Table of Contents

- [Task-Specific Curriculum](#)
- [Teacher-Guided Curriculum](#)
- [Curriculum through Self-Play](#)
- [Automatic Goal Generation](#)
- [Skill-Based Curriculum](#)
- [Curriculum through Distillation](#)
- [Citation](#)
- [References](#)

[Updated on 2020-02-03: mentioning [PCG](#) in the “Task-Specific Curriculum” section.
[Updated on 2020-02-04: Add a new “[curriculum through distillation](#)” section.

It sounds like an impossible task if we want to teach integral or derivative to a 3-year-old who does not even know basic arithmetics. That’s why education is important, as it provides a systematic way to break down complex knowledge and a nice curriculum for teaching concepts from simple to hard. A curriculum makes learning difficult things easier and approachable for us humans. But, how about machine learning models? Can we train our models more efficiently with a curriculum? Can we design a curriculum to speed up learning?

Back in 1993, Jeffrey Elman has proposed the idea of training neural networks with a curriculum. His early work on learning simple language grammar demonstrated the importance of such a strategy: starting with a restricted set of simple data and gradually increasing the complexity of training samples; otherwise the model was not able to learn at all.

Compared to training without a curriculum, we would expect the adoption of the curriculum to expedite the speed of convergence and may or may not improve the final

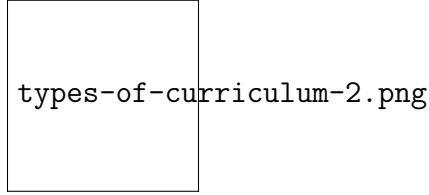


Figure 1: Five types of curriculum for reinforcement learning.

model performance. To design an efficient and effective curriculum is not easy. Keep in mind that, a bad curriculum may even hamper learning.

Next, we will look into several categories of curriculum learning, as illustrated in Most cases are applied to Reinforcement Learning, with a few exceptions on Supervised Learning.

In “The importance of starting small” paper ([Elman 1993](#)), I especially like the starting sentences and find them both inspiring and affecting:

“Humans differ from other species along many dimensions, but two are particularly noteworthy. Humans display an exceptional capacity to learn; and humans are remarkable for the unusually long time it takes to reach maturity. The adaptive advantage of learning is clear, and it may be argued that, through culture, learning has created the basis for a non-genetically based transmission of behaviors which may accelerate the evolution of our species.”

Indeed, learning is probably the best superpower we humans have.

2 Task-Specific Curriculum#

[Bengio, et al. \(2009\)](#) provided a good overview of curriculum learning in the old days. The paper presented two ideas with toy experiments using a manually designed task-specific curriculum:

1. Cleaner Examples may yield better generalization faster.
2. Introducing gradually more difficult examples speeds up online training.

It is plausible that some curriculum strategies could be useless or even harmful. A good question to answer in the field is: *What could be the general principles that make some curriculum strategies work better than others?* The Bengio 2009 paper hypothesized it would be beneficial to make learning focus on “interesting” examples that are neither too hard or too easy.

If our naive curriculum is to train the model on samples with a gradually increasing level of complexity, we need a way to quantify the difficulty of a task first. One idea is to use its minimal loss with respect to another model while this model is pretrained on other tasks ([Weinshall, et al. 2018](#)). In this way, the knowledge of the pretrained model can be transferred to the new model by suggesting a rank of training samples. Fig. 2 shows the effectiveness of the `curriculum` group (green), compared to `control` (random order; yellow) and `anti` (reverse the order; red) groups.

[Zaremba & Sutskever \(2014\)](#) did an interesting experiment on training LSTM to predict the output of a short Python program for mathematical ops without actually

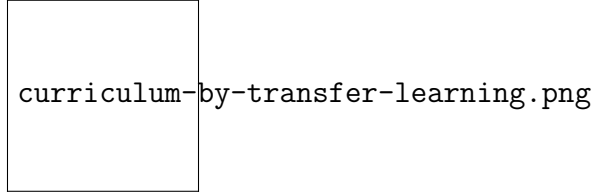


Figure 2: Image classification accuracy on test image set (5 member classes of "small mammals" in CIFAR100). There are 4 experimental groups, (a) 'curriculum': sort the labels by the confidence of another trained classifier (e.g. the margin of an SVM); (b) 'control-curriculum': sort the labels randomly; (c) 'anti-curriculum': sort the labels reversely; (d) 'None': no curriculum. (Image source: [Weinshall, et al. 2018](#))

executing the code. They found curriculum is necessary for learning. The program's complexity is controlled by two parameters, `length` [1, a] and `nesting` [1, b]. Three strategies are considered:

1. Naive curriculum: increase `length` first until reaching a; then increase `nesting` and reset `length` to 1; repeat this process until both reach maximum.
2. Mix curriculum: sample `length` $\sim$ [1, a] and `nesting` $\sim$ [1, b]
3. Combined: naive + mix.

They noticed that combined strategy always outperformed the naive curriculum and would generally (but not always) outperform the mix strategy — indicating that it is quite important to mix in easy tasks during training to *avoid forgetting*.

Procedural content generation (PCG) is a popular approach for creating video games of various levels of difficulty. PCG involves algorithmic randomness and a heavy dose of human expertise in designing game elements and dependencies among them. Procedurally generated levels have been introduced into several benchmark environments for evaluating whether an RL agent can generalize to a new level that it is not trained on ([meta-RL!](#)), such as GVGAI, OpenAI [CoinRun](#) and [Procgen benchmark](#). Using GVGAI, [Justesen, et al. \(2018\)](#) demonstrated that an RL policy can easily overfit to a specific game but training over a simple curriculum that grows the task difficulty together with the model performance helps its generalization to new human-designed levels. Similar results are also found in CoinRun ([Cobbe, et al. 2018](#)). POET ([Wang et al, 2019](#)) is another example for leveraging evolutionary algorithm and procedural generated game levels to improve RL generalization, which I've described in details in my [meta-RL post](#).

To follow the curriculum learning approaches described above, generally we need to figure out two problems in the training procedure:

1. Design a metric to quantify how hard a task is so that we can sort tasks accordingly.
2. Provide a sequence of tasks with an increasing level of difficulty to the model during training.

However, the order of tasks does not have to be sequential. In our Rubik's cube paper ([OpenAI et al, 2019](#)), we depended on *Automatic domain randomization* (**ADR**) to generate a curriculum by growing a distribution of environments with increasing complexity. The difficulty of each task (i.e. solving a Rubik's cube in a set of environments)

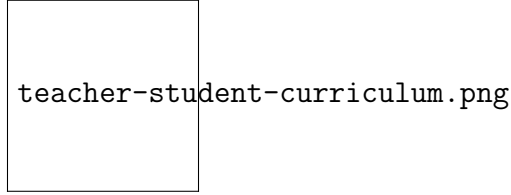


Figure 3: The setup of teacher-student curriculum learning. (Image source: [Matiisen, et al. 2017](#) + my annotation in red.)

depends on the randomization ranges of various environmental parameters. Even with a simplified assumption that all the environmental parameters are uncorrelated, we were able to create a decent curriculum for our robot hand to learn the task.

3 Teacher-Guided Curriculum#

The idea of *Automatic Curriculum Learning* was proposed by [Graves, et al. 2017](#) slightly earlier. It considers a N -task curriculum as an N -armed bandit problem and an adaptive policy which learns to optimize the returns from this bandit.

Two categories of learning signals have been considered in the paper:

1. Loss-driven progress: the loss function change before and after one gradient update. This type of reward signals tracks the speed of the learning process, because the greatest task loss decrease is equivalent to the fastest learning.
2. Complex-driven progress: the KL divergence between posterior and prior distribution over network weights. This type of learning signals are inspired by the MDL principle, “increasing the model complexity by a certain amount is only worthwhile if it compresses the data by a greater amount”. The model complexity is therefore expected to increase most in response to the model nicely generalizing to training examples.

This framework of proposing curriculum automatically through another RL agent was formalized as *Teacher-Student Curriculum Learning* (TSCL; [Matiisen, et al. 2017](#)). In TSCL, a *student* is an RL agent working on actual tasks while a *teacher* agent is a policy for selecting tasks. The student aims to master a complex task that might be hard to learn directly. To make this task easier to learn, we set up the teacher agent to guide the student’s training process by picking proper sub-tasks.

In the process, the student should learn tasks which:

1. can help the student make fastest learning progress, or
2. are at risk of being forgotten.

Note: The setup of framing the teacher model as an RL problem feels quite similar to Neural Architecture Search (NAS), but differently the RL model in TSCL operates on the task space and NAS operates on the main model architecture space.

Training the teacher model is to solve a POMDP problem:

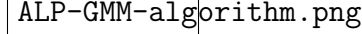

 ALP-GMM-algorithm.png

Figure 4: The algorithm of ALP-GMM (absolute learning progress Gaussian mixture model). (Image source: [Portelas, et al., 2019](#))

- The unobserved s_t is the full state of the student model.
- The observed $o = (x_t^{(1)}, \dots, x_t^{(N)})$ are a list of scores for N tasks.
- The action a is to pick on subtask.
- The reward per step is the score $\Delta r_t = \sum_{i=1}^N x_t^{(i)} - x_{t-1}^{(i)}$ (i.e., equivalent to maximizing the score of all tasks at the end of the episode).

The method of estimating learning progress from noisy task scores while balancing exploration vs exploitation can be borrowed from the non-stationary multi-armed bandit problem — use [-greedy](#), or [Thompson sampling](#).

The core idea, in summary, is to use one policy to propose tasks for another policy to learn better. Interestingly, both works above (in the discrete task space) found that uniformly sampling from all tasks is a surprisingly strong benchmark.

What if the task space is continuous? [Portelas, et al. \(2019\)](#) studied a continuous teacher-student framework, where the teacher has to sample parameters from continuous task space to generate a learning curriculum. Given a newly sampled parameter p , the absolute learning progress (short for ALP) is measured as $\|\text{ALP}_p\| = \|r - r_{\text{old}}\|$, where r is the episodic reward associated with p and r_{old} is the reward associated with p_{old} . Here, p_{old} is a previous sampled parameter closest to p in the task space, which can be retrieved by nearest neighbor. Note that how this ALP score is different from learning signals in [TSCL](#) or [Grave, et al. 2017](#) above: ALP score measures the reward difference between two tasks rather than performance at two time steps of the same task.

On top of the task parameter space, a Gaussian mixture model is trained to fit the distribution of $\|\text{ALP}_p\|$ over p . -greedy is used when sampling the tasks: with some probability, sampling a random task; otherwise sampling proportionally to ALP score from the GMM model.

4 Curriculum through Self-Play#

Different from the teacher-student framework, two agents are doing very different things. The teacher learns to pick a task for the student without any knowledge of the actual task content. What if we want to make both train on the main task directly? How about even make them compete with each other?

[Sukhbaatar, et al. \(2017\)](#) proposed a framework for automatic curriculum learning through **asymmetric self-play**. Two agents, Alice and Bob, play the same task with

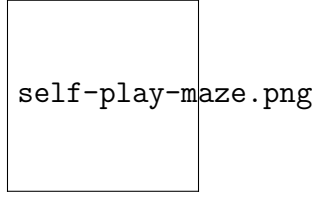


Figure 5: Illustration of the self-play setup when training two agents. The example task is [MazeBase](#): An agent is asked to reach a goal flag in a maze with a light switch, a key and a wall with a door. Toggling the key switch can open or close the door and Turning off the light makes only the glowing light switch available to the agent. (Image source: [Sukhbaatar, et al. 2017](#))

different goals: Alice challenges Bob to achieve the same state and Bob attempts to complete it as fast as he can.

Let us consider Alice and Bob as two separate copies for one RL agent trained in the same environment but with different brains. Each of them has independent parameters and loss objective. The self-play-driven training consists of two types of episodes:

- In the *self-play episode*, Alice alters the state from s_0 to s_t and then Bob is asked to return the environment to its original state s_0 to get an internal reward.
- In the *target task episode*, Bob receives an external reward if he visits the target flag.

Note that since B has to repeat the actions between the same pair of (s_0, s_t) of A, this framework only works in reversible or resettable environments.

Alice should learn to push Bob out of his comfort zone, but not give him impossible tasks. Bob’s reward is set as $R_B = -\gamma t_B$ and Alice’s reward is $R_A = \gamma \max(0, t_B - t_A)$, where t_B is the total time for B to complete the task, t_A is the time until Alice performs the STOP action and γ is a scalar constant to rescale the reward to be comparable with the external task reward. If B fails a task, $t_B = t_{\max} - t_A$. Both policies are goal-conditioned. The losses imply:

1. B wants to finish a task asap.
2. A prefers tasks that take more time of B.
3. A does not want to take too many steps when B is failing.

In this way, the interaction between Alice and Bob automatically builds a curriculum of increasingly challenging tasks. Meanwhile, as A has done the task herself before proposing the task to B, the task is guaranteed to be solvable.

The paradigm of A suggesting tasks and then B solving them does sound similar to the Teacher-Student framework. However, in asymmetric self-play, Alice, who plays a teacher role, also works on the same task to find challenging cases for Bob, rather than optimizes B’s learning process explicitly.

5 Automatic Goal Generation#

Often RL policy needs to be able to perform over a set of tasks. The goal should be carefully chosen so that at every training stage, it would not be too hard or too easy for the current policy. A goal $g \in \mathcal{G}$ can be defined as a set of states S^g and a goal is considered as achieved whenever an agent arrives at any of those states.

The approach of Generative Goal Learning (Florensa, et al. 2018) relies on a **Goal GAN** to generate desired goals automatically. In their experiment, the reward is very sparse, just a binary flag for whether a goal is achieved or not and the policy is conditioned on goal,

$$\pi^*(a_t | s_t, g) = \arg\max_{\pi} \mathbb{E}_{g \sim p_g(\cdot)} R^g(\pi) \quad \text{where } R^g(\pi) = \mathbb{E}_{\pi(\cdot | s_t, g)} \mathbb{1}[\text{exists } t \in [1, \dots, T]: s_t \in S^g]$$

Here $R^g(\pi)$ is the expected return, also equivalent to the success probability. Given sampled trajectories from the current policy, as long as any state belongs to the goal set, the return will be positive.

Their approach iterates through 3 steps until the policy converges:

1. Label a set of goals based on whether they are at the appropriate level of difficulty for the current policy.
 - The set of goals at the appropriate level of difficulty are named **GOID** (short for “Goals of Intermediate Difficulty”).
 $\text{GOID}_i := \{g : R_{\min} \leq R^g(\pi_i) \leq R_{\max}\} \subseteq \mathcal{G}$
 - Here $R_{\min}$ and $R_{\max}$ can be interpreted as a minimum and maximum probability of reaching a goal over T time-steps.
2. Train a Goal GAN model using labelled goals from step 1 to produce new goals
3. Use these new goals to train the policy, improving its coverage objective.

The Goal GAN generates a curriculum automatically:

- Generator $G(z)$: produces a new goal. $\Rightarrow$ expected to be a goal uniformly sampled from GOID set.
- Discriminator $D(g)$: evaluates whether a goal can be achieved. $\Rightarrow$ expected to tell whether a goal is from GOID set.

The Goal GAN is constructed similar to LSGAN (Least-Squared GAN; Mao et al., (2017)), which has better stability of learning compared to vanilla GAN. According to LSGAN, we should minimize the following losses for D and G respectively:

$$\begin{aligned} \mathcal{L}_{\text{LSGAN}}(D) &= \frac{1}{2} \mathbb{E}_{g \sim p_{\text{data}}(g)} [(D(g) - b)^2] + \frac{1}{2} \mathbb{E}_{z \sim p_z(z)} [(D(G(z)) - a)^2] \\ \mathcal{L}_{\text{LSGAN}}(G) &= \frac{1}{2} \mathbb{E}_{z \sim p_z(z)} [(D(G(z)) - c)^2] \end{aligned}$$

where a is the label for fake data, b for real data, and c is the value that G wants D to believe for fake data. In LSGAN paper’s experiments, they used $a=-1$, $b=1$, $c=0$.

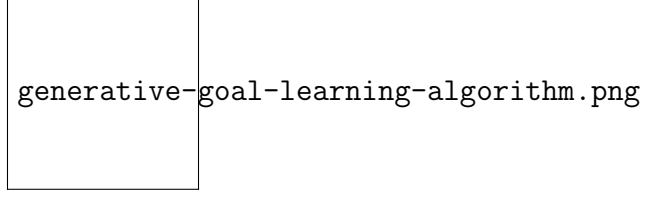


Figure 6: The algorithm of Generative Goal Learning. (Image source: (Florensa, et al. 2018))

The Goal GAN introduces an extra binary flag y_g indicating whether a goal g is real ($y_g = 1$) or fake ($y_g = 0$) so that the model can use negative samples for training:

$$\begin{aligned} \mathcal{L}_{\text{GoalGAN}}(D) &= \frac{1}{2} \mathbb{E}_{g \sim p_{\text{data}}(g)} [(D(g) - b)^2 + (1 - y_g)(D(g) - a)^2] + \frac{1}{2} \mathbb{E}_{z \sim p_z(z)} [(D(G(z)) - a)^2] \\ \mathcal{L}_{\text{GoalGAN}}(G) &= \frac{1}{2} \mathbb{E}_{z \sim p_z(z)} [(D(G(z)) - c)^2] \end{aligned}$$

Following the same idea, [Racaniere & Lampinen, et al. \(2019\)](#) designs a method to make the objectives of goal generator more sophisticated. Their method contains three components, same as generative goal learning above:

- **Solver/Policy** π : In each episode, the solver gets a goal g at the beginning and get a single binary reward R^g at the end.
- **Judge/Discriminator** $D(\cdot)$: A classifier to predict the binary reward (whether goal can be achieved or not); precisely it outputs the logit of a probability of achieving the given goal, $\sigma(D(g)) = p(R^g=1 | g)$, where σ is the sigmoid function.
- **Setter/Generator** $G(\cdot)$: The goal setter takes as input a desired feasibility score $f \in \text{Unif}(0, 1)$ and generates $g = G(z, f)$, where the latent variable z is sampled by $z \sim \mathcal{N}(0, I)$. The goal generator is designed to reversible, so G^{-1} can map backwards from a goal g to a latent $z = G^{-1}(g, f)$

The generator is optimized with three objectives:

1. **Goal validity**: The proposed goal should be achievable by an expert policy. The corresponding generative loss is designed to increase the likelihood of generating goals that the solver policy has achieved before (like in [HER](#)).

- $\mathcal{L}_{\text{val}}$ is the negative log-likelihood of generated goals that have been solved by the solver in the past.

•

$$\mathcal{L}_{\text{val}} = \mathbb{E}_{\{g \sim \text{achieved by solver}\}, \{x \in \text{Uniform}(0, \delta), f \in \text{Uniform}(0, 1)\}} [-\log p(G^{-1}(g + x, f))] \quad \square$$

2. **Goal feasibility**: The proposed goal should be achievable by the current policy; that is, the level of difficulty should be appropriate.

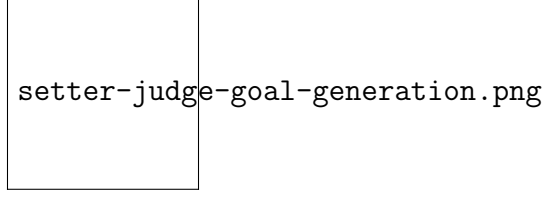


Figure 7: Training schematic for the (a) solver/policy, (b) judge/discriminator, and (c) setter/goal generator models. (Image source: [Racaniere & Lampinen, et al., 2019](#))

- $\mathcal{L}_{\text{feas}}$ is the output probability by the judge model D on the generated goal $G(z, f)$ should match the desired f .

•

$$\mathbb{E}_{z \in \mathcal{N}(0, 1), f \in \text{Uniform}(0, 1)} [D(G(z, f)) - \sigma^{-1}(f)^2] = 0$$

3. **Goal coverage:** We should maximize the entropy of generated goals to encourage diverse goal and to improve the coverage over the goal space.

$$\mathbb{E}_{z \in \mathcal{N}(0, 1), f \in \text{Uniform}(0, 1)} [-\log p(G(z, f))] = 0$$

Their experiments showed complex environments require all three losses above. When the environment is changing between episodes, both the goal generator and the discriminator need to be conditioned on environmental observation to produce better results. If there is a desired goal distribution, an additional loss can be added to match a desired goal distribution using Wasserstein distance. Using this loss, the generator can push the solver toward mastering the desired tasks more efficiently.

6 Skill-Based Curriculum#

Another view is to decompose what an agent is able to complete into a variety of skills and each skill set could be mapped into a task. Let’s imagine when an agent interacts with the environment in an unsupervised manner, is there a way to discover useful skills from such interaction and further build into the solutions for more complicated tasks through a curriculum?

[Jabri, et al. \(2019\)](#) developed an automatic curriculum, **CARML** (short for “Curricula for Unsupervised Meta-Reinforcement Learning”), by modeling unsupervised trajectories into a latent skill space, with a focus on training [meta-RL](#) policies (i.e. can transfer to unseen tasks). The setting of training environments in CARML is similar to [DIAYN](#). Differently, CARML is trained on pixel-level observations but DIAYN operates on the true state space. An RL algorithm π_{θ} , parameterized by θ , is trained via unsupervised interaction formulated as a CMP combined with a learned reward function r . This setting naturally works for the meta-learning purpose, since a customized reward function can be given only at the test time.

CARML is framed as a [variational Expectation-Maximization \(EM\)](#).

(1) **E-Step:** This is the stage for organizing experiential data. Collected trajectories are modeled with a mixture of latent components forming the [basis](#) of *skills*.

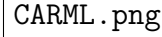


Figure 8: An illustration of CARML, containing two steps: (1) organizing experiential data into the latent skill space; (2) meta-training the policy with the reward function constructed from the learned skills. (Image source: [Jabri, et al 2019](#))

Let z be a latent task variable and q_{ϕ} be a variational distribution of z , which could be a mixture model with discrete z or a VAE with continuous z . A variational posterior $q_{\phi}(z | s)$ works like a classifier, predicting a skill given a state, and we would like to maximize $q_{\phi}(z | s)$ to discriminate between data produced by different skills as much as possible. In E-step, q_{ϕ} is fitted to a set of trajectories produced by π_{θ} .

Precisely, given a trajectory $\tau = (s_1, \dots, s_T)$, we would like to find ϕ such that

$$\max_{\phi} \mathbb{E}_{z \sim q_{\phi}(z)} [\log q_{\phi}(\tau | z)] = \max_{\phi} \mathbb{E}_{z \sim q_{\phi}(z)} [\log \sum_{s_i \in \tau} q_{\phi}(s_i | z)]$$

A simplifying assumption is made here to ignore the order of states in one trajectory.

(2) **M-Step:** This is the stage for doing meta-RL training with π_{θ} . The learned skill space is considered as a training task distribution. CARML is agnostic to the type of meta-RL algorithm for policy parameter updates.

Given a trajectory τ , it makes sense for the policy to maximize the mutual information between τ and z , $I(\tau; z) = H(\tau) - H(\tau | z)$, because:

- maximizing $H(\tau)$ $\Rightarrow$ diversity in the policy data space; expected to be large.
- minimizing $H(\tau | z)$ $\Rightarrow$ given a certain skill, the behavior should be restricted; expected to be small.

Then we have,

$$\begin{aligned} I(\tau; z) &= \mathcal{H}(z) - \mathcal{H}(z | s_1, \dots, s_T) \\ &\geq \mathbb{E}_{s \in \tau} [\mathcal{H}(z) - \mathcal{H}(z | s)] \\ &\quad \text{\scriptsize{; discard the order of states.}} \\ &= \mathbb{E}_{s \in \tau} [\mathcal{H}(s) - \mathcal{H}(s | z)] \\ &\quad \text{\scriptsize{; by definition of MI.}} \\ &= \mathbb{E}_{z \sim q_{\phi}(z), s \sim \pi_{\theta}(s|z)} [\log q_{\phi}(s|z) - \log \pi_{\theta}(s)] \\ &\quad \text{\scriptsize{; assume learned marginal distr. matches policy.}} \end{aligned}$$

We can set the reward as $\log q_{\phi}(s | z) - \log q_{\phi}(s)$, as shown in the red part in the equation above. In order to balance between task-specific exploration (as in red below) and latent skill matching (as in blue below), a parameter $\lambda \in [0, 1]$ is added. Each realization of $z \sim q_{\phi}(z)$ induces a reward function $r_z(s)$ (remember that reward + CMP $\Rightarrow$ MDP) as follows:

$$r_z(s) = \lambda \log q_{\phi}(s|z) - \log q_{\phi}(s) + (1 - \lambda) \log \frac{q_{\phi}(s|z)}{q_{\phi}(z)}$$

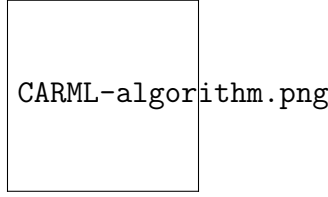


Figure 9: The algorithm of CARML. (Image source: [Jabri, et al 2019](#))

$$q_{\backslash\mathrm{phi}}(s|z) - \log q_{\backslash\mathrm{phi}}(s|z) - \log q_{\backslash\mathrm{phi}}(z) + \log q_{\backslash\mathrm{phi}}(z|s) \quad \&= (\lambda - 1) \\ \log \color{red}{q_{\backslash\mathrm{phi}}(s|z)} + \color{blue}{\log q_{\backslash\mathrm{phi}}(z|s)} + C \quad \end{aligned} \quad \mathbb{S}$$

Learning a latent skill space can be done in different ways, such as in Hausman, et al. 2018. The goal of their approach is to learn a task-conditioned policy, $\pi(a \mid s, t^{\{i\}})$, where $t^{\{i\}}$ is from a discrete list of N tasks, $\mathcal{T} = \{t^{\{1\}}, \dots, t^{\{N\}}\}$. However, rather than learning N separate solutions, one per task, it would be nice to learn a latent skill space so that each task could be represented in a distribution over skills and thus skills are *reused between tasks*. The policy is defined as $\pi_{\theta}(a \mid s, t) = \int \pi_{\theta}(a \mid z, s, t) p_{\phi}(z \mid t) \mathrm{d}z$, where π_{θ} and p_{ϕ} are policy and embedding networks to learn, respectively. If z is discrete, i.e. drawn from a set of K skills, then the policy becomes a mixture of K sub-policies. The policy training uses SAC and the dependency on z is introduced in the entropy term.

7 Curriculum through Distillation#

[I was thinking of the name of this section for a while, deciding between cloning, inheritance, and distillation. Eventually, I picked distillation because it sounds the coolest B-)]

The motivation for the **progressive neural network** (Rusu et al. 2016) architecture is to efficiently transfer learned skills between different tasks and in the meantime avoid catastrophic forgetting. The curriculum is realized through a set of progressively stacked neural network towers (or “columns”, as in the paper).

A progressive network has the following structure:

1. It starts with a single column containing L layers of neurons, in which the corresponding activation layers are labelled as $h^{(1)}_i, i=1, \dots, L$. We first train this single-column network for one task to convergence, achieving parameter config $\theta^{(1)}$.
2. Once switch to the next task, we need to add a new column to adapt to the new context while freezing $\theta^{(1)}$ to lock down the learned skills from the previous task. The new column has activation layers labelled as $h^{(2)}_i, i=1, \dots, L$, and parameters $\theta^{(2)}$.
3. Step 2 can be repeated with every new task. The i -th layer activation in the k -th column depends on the previous activation layers in all the existing columns:

$$h^{\wedge}\{(k)\}_i = f(W^{\wedge}\{(k)\}_i h^{\wedge}\{(k)\}_{-i-1}) + \sum_{j < k} U_i^{\wedge}\{(k:j)\} h^{\wedge}\{(j)\}_{-i-1})$$

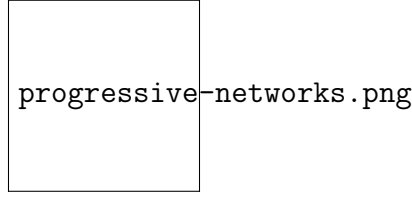


Figure 10: The progressive neural network architecture. (Image source: [Rusu, et al. 2017](#))

where $W^{(k)}_i$ is the weight matrix of the layer i in the column k ; $U^{(k:j)}_{i-1}$, $j < k$ are the weight matrices for projecting the layer $i-1$ of the column j to the layer i of column k ($j < k$). The above weights matrices should be learned. $f(\cdot)$ is a non-linear activation function by choice.

The paper experimented with Atari games by training a progressive network on multiple games to check whether features learned in one game can transfer to another. That is indeed the case. Though interestingly, learning a high dependency on features in the previous columns does not always indicate good transfer performance on the new task. One hypothesis is that features learned from the old task might introduce biases into the new task, leading to policy getting trapped in a sub-optimal solution. Overall, the progressive network works better than only fine-tuning the top layer and can achieve similar transfer performance as fine-tuning the entire network.

One use case for the progressive network is to do sim2real transfer ([Rusu, et al. 2017](#)), in which the first column is trained in simulator with a lot of samples and then the additional columns (could be for different real-world tasks) are added and trained with a few real data samples.

[Czarnecki, et al. \(2018\)](#) proposed another RL training framework, **Mix & Match** (short for **M&M**) to provide curriculum through coping knowledge between agents. Given a sequence of agents from simple to complex, $\pi_1, \dots, \pi_K$, each parameterized with some shared weights (e.g. by shared some lower common layers). M&M trains a mixture of agents, but only the final performance of the most complex one π_K matters.

In the meantime, M&M learns a categorical distribution $c \sim \text{Categorical}(1, \dots, K \text{ } \alpha)$ with pmf $p(c=i) = \alpha_i$ probability to pick which policy to use at a given time. The mixed M&M policy is a simple weighted sum: $\pi_{\text{mm}}(a \text{ } s) = \sum_{i=1}^K \alpha_i \pi_i(a \text{ } s)$. Curriculum learning is realized by dynamically adjusting α_i , from $\alpha_K=0$ to $\alpha_K=1$. The tuning of α can be manual or through [population-based training](#).

To encourage cooperation rather than competition among policies, besides the RL loss $\mathcal{L}_{\text{RL}}$, another [distillation](#)-like loss $\mathcal{L}_{\text{mm}}(\theta)$ is added. The knowledge transfer loss $\mathcal{L}_{\text{mm}}(\theta)$ measures the KL divergence between two policies, $\text{propto } D_{\text{KL}}(\pi_i(\cdot \text{ } s) \parallel \pi_j(\cdot \text{ } s))$ for $i < j$. It encourages complex agents to match the simpler ones early on. The final loss is $\mathcal{L} = \mathcal{L}_{\text{RL}}(\theta \text{ } \pi_{\text{mm}}) + \lambda \mathcal{L}_{\text{mm}}(\theta)$.

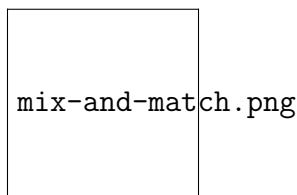


Figure 11: The Mix & Match architecture for training a mixture of policies. (Image source: [Czarnecki, et al., 2018](#))

8 Citation#

Cited as:

Weng, Lilian. (Jan 2020). Curriculum for reinforcement learning. Lil'Log.
<https://lilianweng.github.io/posts/2020-01-29-curriculum-rl/>.

Or

```
@article{weng2020curriculum,
  title   = "Curriculum for Reinforcement Learning",
  author  = "Weng, Lilian",
  journal = "lilianweng.github.io",
  year    = "2020",
  month   = "Jan",
  url     = "https://lilianweng.github.io/posts/2020-01-29-curriculum-rl/"
}
```

9 References#

- [1] Jeffrey L. Elman. “[Learning and development in neural networks: The importance of starting small.](#)” *Cognition* 48.1 (1993): 71-99.
- [2] Yoshua Bengio, et al. “[Curriculum learning.](#)” ICML 2009.
- [3] Daphna Weinshall, Gad Cohen, and Dan Amir. “[Curriculum learning by transfer learning: Theory and experiments with deep networks.](#)” ICML 2018.
- [4] Wojciech Zaremba and Ilya Sutskever. “[Learning to execute.](#)” arXiv preprint arXiv:1410.4615 (2014).
- [5] Tabet Matiisen, et al. “[Teacher-student curriculum learning.](#)” *IEEE Trans. on neural networks and learning systems* (2017).
- [6] Alex Graves, et al. “[Automated curriculum learning for neural networks.](#)” ICML 2017.
- [7] Remy Portelas, et al. [Teacher algorithms for curriculum learning of Deep RL in continuously parameterized environments.](#) CoRL 2019.
- [8] Sainbayar Sukhbaatar, et al. “[Intrinsic Motivation and Automatic Curricula via Asymmetric Self-Play.](#)” ICLR 2018.
- [9] Carlos Florensa, et al. “[Automatic Goal Generation for Reinforcement Learning Agents](#)” ICML 2019.
- [10] Sebastien Racaniere & Andrew K. Lampinen, et al. “[Automated Curriculum through Setter-Solver Interactions](#)” ICLR 2020.

- [11] Allan Jabri, et al. “Unsupervised Curricula for Visual Meta-Reinforcement Learning” NeurIPS 2019.
- [12] Karol Hausman, et al. “Learning an Embedding Space for Transferable Robot Skills” ICLR 2018.
- [13] Josh Merel, et al. “Reusable neural skill embeddings for vision-guided whole body movement and object manipulation” arXiv preprint arXiv:1911.06636 (2019).
- [14] OpenAI, et al. “Solving Rubik’s Cube with a Robot Hand.” arXiv preprint arXiv:1910.07113 (2019).
- [15] Niels Justesen, et al. “Illuminating Generalization in Deep Reinforcement Learning through Procedural Level Generation” NeurIPS 2018 Deep RL Workshop.
- [16] Karl Cobbe, et al. “Quantifying Generalization in Reinforcement Learning” arXiv preprint arXiv:1812.02341 (2018).
- [17] Andrei A. Rusu et al. “Progressive Neural Networks” arXiv preprint arXiv:1606.04671 (2016).
- [18] Andrei A. Rusu et al. “Sim-to-Real Robot Learning from Pixels with Progressive Nets.” CoRL 2017.
- [19] Wojciech Marian Czarnecki, et al. “Mix & Match – Agent Curricula for Reinforcement Learning.” ICML 2018.

- [Reinforcement-Learning](#)
- [Generative-Model](#)
- [Meta-Learning](#)

The Transformer Family

Self-Supervised Representation Learning

© 2025 Lil'Log Powered by Hugo & PaperMod

10 Citation

Please cite this work as:

Weng, Lilian. “[Title](#)”. Lil 'Log (May 2025). <https://lilianweng.github.io/posts/2020-01-29-curriculum-rl/>

Or use the BibTeX citation:

```
@article{weng2025,
  title = {},
  author = {Weng, Lilian},
  journal = {lilianweng.github.io},
  year = {2025}, month = {May},
  url = {}
}
```


11 References

Tags: [tag1](#) [tag2](#)

[⟨ Title Title ⟩](#)

Share this article: [Twitter](#) | [LinkedIn](#) | [Reddit](#) | [Facebook](#) | [WhatsApp](#) | [Telegram](#)